

JIT-Accelerated DP-SGD in PyTorch

Problem Statement

Differentially private SGD (DP-SGD) is critical for AI ethics, ensuring models do not leak individual data. Prior work shows that naively trained deep models can regurgitate private data like Social Security numbers, so DP-SGD is needed to prevent such leaks. Unfortunately, DP-SGD typically incurs huge slowdowns compared to standard SGD. For example, Rodriguez Beltran et al. report that unoptimized DP-SGD can be 3–8x slower than non-private training on large vision models. This overhead makes privacy costly and hinders adoption. As Subramani et al. note, exploiting JIT compilation and vectorization can “mostly eliminate the significant running time overhead” of DP-SGD. Our problem statement is: **can we extend these JIT-based speedups to PyTorch?** We aim to implement a JIT-accelerated DP-SGD pipeline in PyTorch so that private training can run nearly as fast as non-private SGD, closing the performance gap noted above.

Relation to Existing Work

Recent research highlights the performance gap in DP training. Subramani et al. showed that in JAX and TensorFlow 2 with XLA, DP-SGD can run as fast as non-private SGD by using JIT compilation and batching. In their experiments, JAX was by far the most performant implementation of DP-SGD, and their optimized TensorFlow implementation (with XLA) also nearly matched non-private speed. In contrast, PyTorch’s main DP library, Opacus, uses Python forward/backward hooks to compute per-sample gradients. While Opacus provides a user-friendly PrivacyEngine and vectorized operations, its implementation still incurs substantial overhead on large models (e.g. vision transformers) because it cannot fuse operations into static kernels.

PyTorch 2.x introduced new JIT tools (e.g. `torch.compile()`) that promise 1.5–2x speedups by compiling PyTorch code into optimized kernels. However, Beltran et al. (2024) attempted to apply `torch.compile` to DP-SGD and found no speed improvement. PyTorch’s compiler simply fell back to pre-optimized CUDA kernels and ignored the custom Opacus hooks. As they note, compiling a private model continued execution without compiling the DP-specific code. In other words, out-of-the-box PyTorch JIT does not yet accelerate DP-SGD. To our knowledge, no published work has successfully integrated DP-SGD with PyTorch’s JIT or functional APIs for end-to-end compiled execution. This project will fill that gap by designing a PyTorch-native DP-SGD that truly leverages JIT/vectorization.

Proposed Methodology

To achieve JIT acceleration in PyTorch, we will redesign DP-SGD using PyTorch’s latest compilation and functional tools.

Specifically, we plan to:

- **Implement per-sample gradients with `functorch` or `torch.func.vmap`.**
Instead of using hooks, we will use PyTorch’s functional API (e.g. `torch.func.vmap` over samples combined with `torch.func.grad`) to compute gradients for each example in a batch. This creates a single differentiable graph for the batch.

- **Apply `torch.compile()` to the training loop.**
We will wrap the core DP-SGD update (forward, loss, backward, clipping, noise) inside a `@torch.compile` region so that the entire computation can be fused. This should turn the Python loop and tensor operations into optimized kernels.
- **Design a PyTorch-friendly API.**
We will build or extend a “PrivacyEngine” interface that integrates the JIT-enabled DP-SGD. For example, a compiled function could take model parameters, input batch, and output clipped noisy gradients. The API will mimic Opacus (e.g. attaching to an optimizer) so users need minimal code changes.
- **Benchmark on standard tasks.** We will test on tasks like image classification (MNIST, CIFAR) and NLP (IMDb sentiment) to measure runtime and accuracy. Benchmarks will compare our JIT-PyTorch DP-SGD against baseline Opacus and against the JAX/XLA implementations from Subramani *et al.*. We will record wall-clock time per epoch and GPU memory.
- **Validate privacy accounting.** We will verify that our implementation yields the same ϵ, δ as standard DP-SGD. In particular, we will ensure that any use of compiled kernels or lower precision does not violate DP guarantees (e.g. by rounding issues).

Each of these steps will be carefully documented. We will use PyTorch versions (2.x) and relevant libraries (TorchDynamo/TorchInductor) on GPU hardware to ensure realistic speed measurements.

Expected Outcomes

- **A working PyTorch DP-SGD implementation with JIT acceleration.** The core deliverable will be a PyTorch module or library that can train models with DP-SGD using just-in-time compilation. We expect to release this code as open-source with examples, allowing others to easily switch on DP training with JIT.
- **Empirical performance results.** We anticipate substantial speed-ups over existing PyTorch DP. Ideally, private training will approach the speed of non-private SGD as in JAX. Even a 2–3x improvement would be significant. Our benchmarks will quantify how much overhead remains (or is eliminated) at various batch sizes and model sizes. We will cite results similarly to prior work.
- **Analysis and documentation.** The project report will include graphs of runtime, memory usage, and accuracy. We will analyze when JIT helps (e.g. large batch sizes, large models) and where it may have limits (e.g. dynamic batch sizes causing recompilation). Any novel techniques (such as custom kernels or static graph tricks) will be documented.
- **Ethical considerations.** By making DP-SGD faster and more accessible, our work encourages the adoption of privacy-preserving training. We will explicitly discuss that faster DP should not be misused to hide privacy-violations; for example, we will verify that using mixed precision or compilation does not break the formal DP guarantee.

If successful, our project will demonstrate that PyTorch can also benefit from JIT-based speedups in DP training.

References:

1. Enabling Fast Differentially Private SGD via Just-in-Time Compilation and Vectorization(2021)
2. Towards Efficient and Scalable Implementation of Differentially Private Deep Learning (2024)